

(../index.html)

常见问题 (../faq.html)

用户手册 (index.html)

API (../api.html)

视频 (../videos.html)

表现 (../performance.html)

参考 (../references.html)

支持 (../contact.html)

介绍 (index.html)

Obi 7 中的新功能 (whatsnew.html) **!**

从 Obi 6 升级 (upgrading.html) **新品!**

设置 (setup.html)

工作原理 (convergence.html)

更新循环 (updateloop.html)

建筑学 (bigpicture.html)

求解器 (obisolver.html)

后端 (backends.html)

演员 (actors.html)

粒子渲染 (particlerendering.html)

性能提示 (performancetips.html)

故障排除 (troubleshooting.html)

约束

碰撞 (collisions.html)

碰撞材料 (collisionmaterials.html)

表面碰撞 (surfacecollisions.html)

距离场 (distancefields.html)

力区 (forcezones.html)

附件 (attachments.html)

引脚约束 (pinconstraints.html)

缝纫机 (stitcher.html)

针迹约束 (stitchconstraints.html)

距离约束 (distanceconstraints.html)

体积限制 (volumeconstraints.html)

空气动力学约束 (aerodynamics.html)

弯曲约束 (bendingconstraints.html)

系绳约束 (tetherconstraints.html)

蒙皮约束 (skinconstraints.html)

形状匹配约束 (shapematchingconstraints.html)

拉伸/剪切约束 (stretchshearconstraints.html)

弯曲/扭曲约束 (bendtwistconstraints.html)

链约束 (chainconstraints.html)

腰带

布料设置 (clothsetup.html)

布料蓝图编辑器 (clothblueprinteditor.html)

布料渲染 (clothrenderingmodes.html)

撕裂 (clothtearing.html)

角色服装 (characterclothing.html)

腰带

绳索/杆设置 (ropesetup.html)

绳索渲染 (roperenderingmodes.html)

改变绳索长度 (ropecursor.html)

针孔 (ropepinhole.html)

绳索撕裂 (ropetearing.html)

角色骨骼 (bonesetup.html)

奥比流体

发射器 (emitters.html)

发射器蓝图 (emittermaterials.html)

流体渲染 (fluidrendering.html)

平流 (泡沫) (particleadvection.html)

扩散 (particlediffusion.html)

Obi 软体

软体设置 (softbodysetup.html)

软体蓝图编辑器 (softbodyblueprinteditor.html)

软体渲染 (softbodyskinner.html)

角色软体 (charactersoftbody.html)

脚本 (scripting.html)

脚本求解器 (scriptingsolver.html)

空间查询 (scriptingqueries.html)

脚本冲突 (scriptingcollisions.html)

脚本平流 (scriptingadvection.html)

脚本附件 (scriptingattachments.html)

脚本约束 (scriptingconstraints.html)

粒子 API (scriptingparticles.html)

剧本演员 (scriptingactors.html)

脚本绳索 (scriptingropes.html)

Obi 的工作原理

版本 7.0 ▾

本页信息将为您提供关于 Obi 物理模拟方法的宝贵见解，但它更像是一场游戏物理讲座，而非手册的一部分。虽然并非使用 Obi 的必要条件，但它可以帮助您在调整参数时做出明智的决策，从而更有效地使用它，因此您应该考虑阅读它。

您不会后悔，回报是巨大的！

Obi 将一切建模为一组**粒子**（自由移动的物质块）和**约束**（控制粒子行为的规则），使用一种称为**基于位置的动力学**（简称**PBD**）的模拟范式。

在PBD中，力和速度在模拟中起着次要的作用，取而代之的是位置。与传统的基于速度的模拟方法相比，它的主要优势在于**无条件稳定**。

稳定

我们所说的“无条件稳定”究竟是什么意思呢？我们指的是模拟不会因为游戏中常见的特定情况而崩溃。但为什么大多数物理引擎偶尔会导致物体出现痉挛性行为或自行消亡，而 Obi 却应该对此免疫呢？为了找到答案，让我们构建一个简单的传统物理引擎并进行测试：

物理模拟涉及计算某些对象**属性**（例如位置或速度）如何**随时间变化**。时间是一个连续变量，但计算机是**数字化的**：这意味着它们无法表示真正连续的数据，必须将其分割成有限大小的小块。因此，在执行物理模拟时，我们将时间划分为**时间步**，类似于在渲染过程中将时间划分为**帧**。（有关模拟和渲染如何协同工作的详细信息，请参阅更新循环）。(updateloop.html)

有了这些知识，让我们根据物体的速度，将其从一个时间步移动到下一个时间步。要做到这一点，我们只需要记住**速度是位置随时间变化的速率**：

```
位置 += 速度 * 时间步长；
```

很简单。如果我们也想更新它的速度（例如，由于某些外部加速度，如重力），我们只需要记住**加速度是速度随时间的变化率**：

```
速度 += 重力加速度 * 时间步长；  
位置 += 速度 * 时间步长；
```

就这样：**我们刚刚构建了一个物理引擎，它能够使用数值积分来模拟一个物体。现在到了有趣的部分：我们将使用弹簧将这个物体连接到空间中的某个点。**

弹簧施加的力就是**刚度 x 变形**。变形是弹簧当前长度与静止长度之间的差值。

为了简单起见，我们还假设物体的质量为 1。由于**力 = 质量 x 加速度**，力等于加速度，**加速度 = 刚度 x 变形**。

```
currentSpring = 附加点 - 位置；  
变形 = 当前弹簧 - 静止长度 * 当前弹簧.标准化；  
弹簧加速度 = 刚度 * 变形；  
  
速度+ = （重力加速度 + 弹簧加速度） * 时间步长；  
位置 += 速度 * 时间步长；
```

让我们使用**0.016**秒的 timestepDuration 和**200** 的刚度值，然后在循环中执行此代码，在每个时间步结束时绘制弹簧和物体：



这是一个漂亮的钟摆。现在让我们把弹簧的硬度调高到 **16000**，希望能消除弹性：



哎呀！如此惨败的原因在于，我们超硬弹簧引起的加速度非常大。由于速度在每个时间步长内仅更新一次，模拟无法对速度的快速变化做出反应：物体*快速*地向目标位置移动，但却**飞快地超过了**目标位置。当我们到达下一个时间步长时，弹簧的**拉伸程度更大**，因此它会尝试通过对物体施加更大的加速度来回到其静止长度：随着每个时间步长的推移，情况会变得更糟，模拟会随着时间的**推移而发散**，变得不稳定。

为了避免过度调整，我们需要更频繁地更新模拟，使用**0.005 秒的时间步长**：



这很不妙，因为较大的速度（在游戏中很常见）会迫使我们更频繁地更新物理以保持稳定性，这会影响**性能**。更糟糕的是，使用较小的时间步长并不能**保证稳定性**：它只是在模拟崩溃之前给了我们更多的余地。

如果我们无法计算许多小的时间步长，就需要手动限制游戏中的质量和力，以确保不会出现过大的速度，物体也不会爆炸。当模拟的复杂性增加，并且玩家能够与之互动时，情况就变成了一场噩梦，因为计算人为限制不再容易，而解决稳定性问题需要我们使用非常小的时间步长。以下多摆就是这种情况：

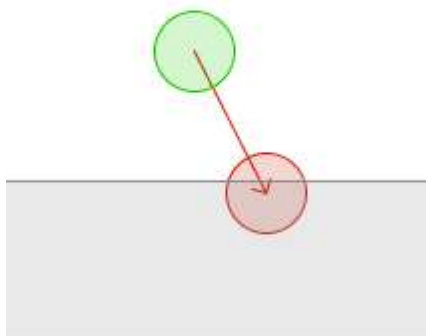


有没有一种简单、稳健的替代方案，无论时间步长和模拟复杂度如何，都能始终保持良好的性能？幸运的是，确实存在：

基于位置的动力学

在每个时间步开始时，Obi 都会根据粒子的**速度**和模拟的**时间步长**，**预测每个粒子的一个新的暂定位置**。这个暂定位置可能违反了许多约束：它可能位于对撞机内部，或者远离弹簧所需的位置。

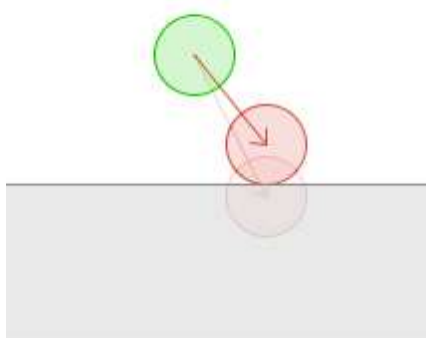
```
// 应用重力等外部加速度，然后计算暂定位置：  
速度 += 加速度 * 时间步长；  
暂定位置 = 位置 + 速度 * 时间步长；
```



粒子从其起始位置（绿色）前进到其暂定位置（红色），该位置是通过速度计算得出的。遗憾的是，这个暂定位置与一个碰撞器相交，所以我们无法立即将粒子推进到那里。

需要**调整**该位置，使其满足影响该粒子的**约束**所施加的所有条件。然后，我们计算粒子从当前位置移动到暂定位置的**速度**：

```
// 调整暂定位置：  
暂定位置 = 有效位置；  
  
// 计算到达那里的速度：  
速度 = (tentativePosition - 位置) / timestepDuration;
```

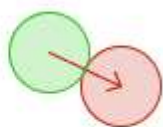


暂定位置（红色）经过修正，以满足碰撞约束：没有粒子可以位于碰撞器内部。

最后，我们将粒子移动到新的位置：

```
// 移动粒子：  
位置 = 暂定位置；
```

我们每个时间步都执行这三个步骤：预测暂定位置、修正暂定位置、前进到修正位置。推进多个时间步，我们会得到如下结果：



只有绿色位置才会渲染到屏幕上，因此我们可以看到遵循物理定律的粒子的平滑动画。

使用这种方法，我们可以将弹簧表述为**位置约束**而不是力，只需将物体移动到不会使弹簧变形的位
置：我们只调整位置，速度是根据位置计算的，因此不可能发生过冲。

因此，无论我们选择何种时间步长、用户输入的剧烈程度或约束的刚性程度，模拟**都不会崩溃**——它们会自动保持与时间步长允许的刚性程度一致。使用较长的时间步长将产生更柔和的约束，使模拟能够轻松处理而不会崩溃。



即使用户猛烈摇晃顶部的灰色颗粒，链条仍然保持非常坚硬和稳定。

Obi 还允许通过为每个约束分配一个柔顺性值来指定所需的最大刚度。柔顺性是刚度的倒数，因此零柔顺性表示“在当前时间步长内尽可能刚度”。柔顺约束表示软约束，非柔顺约束表示刚约束。

那么，这种方法在实践中是否无条件稳定？**是的，除非与并非无条件稳定的 Unity 刚体结合使用。**Unity 的刚体引擎采用经典的基于力/速度的方法。这意味着与刚体的碰撞或附着可能不稳定，因为 Obi 计算和施加的力可能无法通过刚体模拟进行稳健处理。这可能需要您降低 Unity 的固定时间步长设置（位于“项目设置”→“时间”）和/或减少 Obi 使用的子步长数量，以使两个引擎以相似的频率执行模拟。

迭代次数

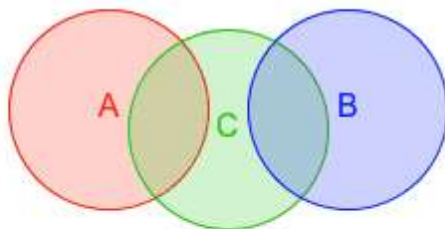
Obi 中的每个约束都将一组粒子和（可选）一些有关“外部”世界的信息作为输入：碰撞器、刚体、风、大气压力等。然后，它会修改粒子的位置，使它们满足某个条件。

有时，强制执行一个约束可能会违反另一个约束，这使得找到一个同时满足所有约束的暂定位置变得困难。Obi 将尝试以**迭代**方式找到所有约束的全局解。每次迭代，我们都会得到一个**更好的解**，更接近同时满足**所有约束**。

Obi 可以通过两种方式迭代所有约束：**顺序或并行**。在**顺序**模式下，会先评估每个约束，并**立即**应用由此产生的对粒子位置的调整，然后再前进到下一个约束。因此，约束的迭代顺序会对最终结果产生轻微影响。在**并行**模式下，会在第一轮中评估所有约束。然后在第二轮中，对每个粒子的调整进行平均并应用。因此，并行模式与**顺序无关**，但它接近真实解的速度较慢。

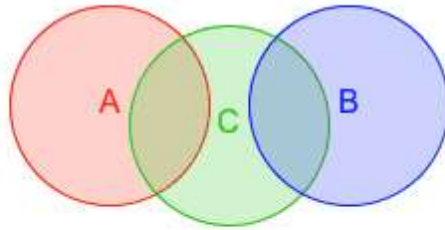
在以下动画中，三个粒子（**A、B 和 C**）产生两个碰撞约束，然后对其进行求解。这一切都发生在一个模拟步骤中：

START!



以顺序模式解决两个碰撞约束。

START!



两个碰撞约束以并行模式求解。需要注意的是，需要 6 次并行迭代才能达到 3 次连续迭代即可获得的结果。

每次额外的迭代都会使你的模拟更接近真实值，但也会略微降低性能。因此，对于特定类型的约束，迭代次数就像一个滑块，在**性能（少量迭代）**和**质量（大量迭代）**之间做出选择。

一旦此时间步的所有迭代都完成，并且粒子位置也已调整，就会计算出**新的速度**，然后我们就可以进行下一个迭代了。以下动画展示了多个时间步的完整过程：



每一步结束时仅渲染绿色位置。红色位置是暂定位置，最初仅使用步骤开始时的粒子速度计算，然后每一步都经过多次迭代进行优化。只有在迭代完成后，粒子才能移动到暂定的（现已调整好的）最终位置。

在大多数情况下，**较大的模拟**（包含更多粒子和约束的模拟，例如长/高分辨率绳索）需要较小的时间步长或更高的迭代次数才能呈现出刚性。预算不足几乎总是会表现为某种不必要的柔软度/弹性，具体取决于哪些约束无法完全满足：

- Stretchy cloth/ropes if **distance constraints** could not be met.
- Bouncy, compressible fluid if **density constraints** could not be met.
- Weak, soft collisions if **collision constraints** could not be met, and so on.

The most effective way to increase perceived stiffness and overall simulation quality is to use a **shorter timestep duration**. This can be accomplished either by increasing the amount of **substeps** in your solver (obisolver.html), or decreasing Unity's *fixed timestep* (found in ProjectSettings → Time). Intuitively speaking, taking smaller steps when advancing the simulation causes the tentative position calculated at the beginning of each step to be **closer to the valid position we started from**. This way, we need less iterations to arrive at a new valid position.



With a timestep of 0.1 ms, 1 iteration per step, the rope is very stretchy.



Increasing iterations to 10 keeps it taut, but quality is not that good as motion is dampened.



With a timestep of 0.01 ms (10 substeps), only one iteration is enough to entirely eliminate stretching and achieve more lively dynamics.

On the flip side, using few iterations and/or a large timestep is absolutely fine when you're looking for stretchy behavior. Instead of increasing constraint compliance (elasticity), **simply spend less substeps/iterations**: the naked eye cannot tell apart a cheap, low-quality simulation from an expensive, high-quality simulation of a very elastic object.

Unlike other engines, Obi allows you to set the amount of iterations spent in each type of constraint **individually**. Each one will affect the simulation in a different way, depending on what the specific type of constraint does, so you can really fine tune your simulation.

The quality improvement you get by reducing the timestep duration is greater than you'd get by using more iterations, and they both have similar performance cost.

For this reason, it's **best to start by setting all constraint iterations to 1** and the solver's **substeps** to a value that yields acceptable quality. Once you're happy with the amount of substeps, you may want to spend a few extra iterations on specific constraint types that could use some extra stiffness. You should never need to use more than 2 or 3 iterations for any constraint type.

Takeaway message: If your rope/cloth/softbody/fluid is too stretchy or bouncy, you should try the following:

- Increasing the amount of **substeps** in the solver.
- Increasing the amount of **constraint iterations in the solver**.
- Decreasing Unity's **fixed timestep**.

Mass and constraints

When two bodies are involved in a constraint of any kind (a collision constraint, a distance constraint, etc), the amount of displacement applied to each one depends on their **mass**: the heavier of the two will be displaced a smaller distance. The reason for this is that **Force = Mass x Acceleration**. As a result, the change in velocity caused by a force is inversely proportional to the mass of the object: **Acceleration** (change in velocity) = **Force / Mass**.

In Obi, you can set the mass of cloth (clothblueprinteditor.html) and softbody (softbodyblueprinteditor.html) particles using their blueprint editor, the mass of ropes and rods is set on a per-control point basis using the path editor (ropesetup.html), and the mass of fluids depends on their density (emittermaterials.html).

Why bother talking about something as ordinary as mass? Because in the context of physics simulation, there's at least **3 common pitfalls** associated with it:

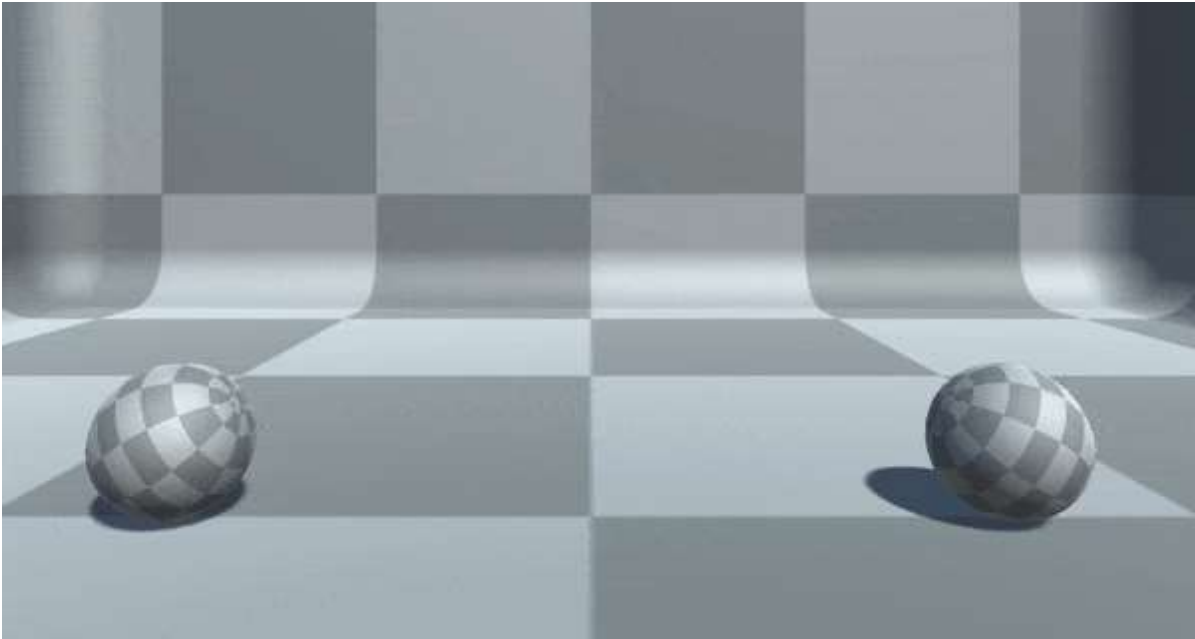
#1. Heavier objects do not fall faster

Generally speaking, gravity is a force of attraction between bodies, the magnitude of which depends on the object's masses and the distance between their centers of mass. However when one of the objects is a **planet** and the other is an comparatively **light** object that **lies on its surface** (a car, an apple, you, me), their mass ratio and the distance between them can be considered constant: no matter how heavy an object is, it's not going to skew the balance in its favor against a *planet*, and no matter how high it goes within the planet's atmosphere the distance between it and the center of the planet won't significantly change.

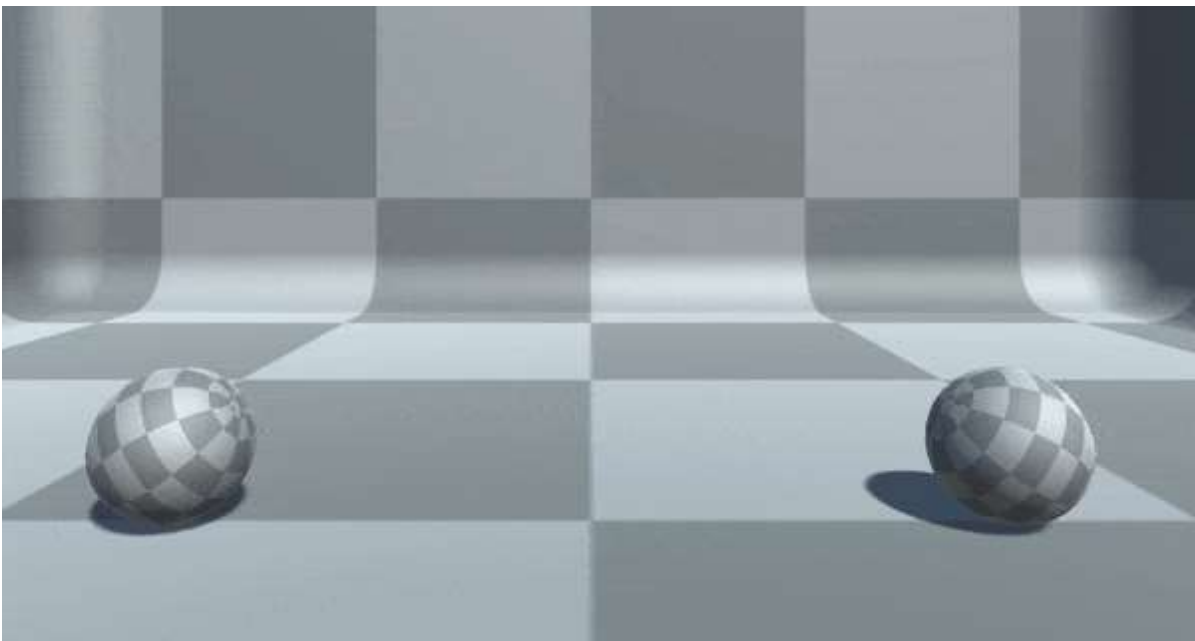
So for **everyday, human-scale affairs** taking place on the surface of a planet, gravity is considered an **acceleration** whose magnitude only depends on the planet you're on (in Earth's case, -9.81 m/s^2): the mass of objects does not affect how fast they fall. Air resistance can make lighter objects fall slower though - in Obi you can model this using aerodynamic constraints (aerodynamics.html) for ropes and cloth.

#2. The absolute mass of objects does not matter

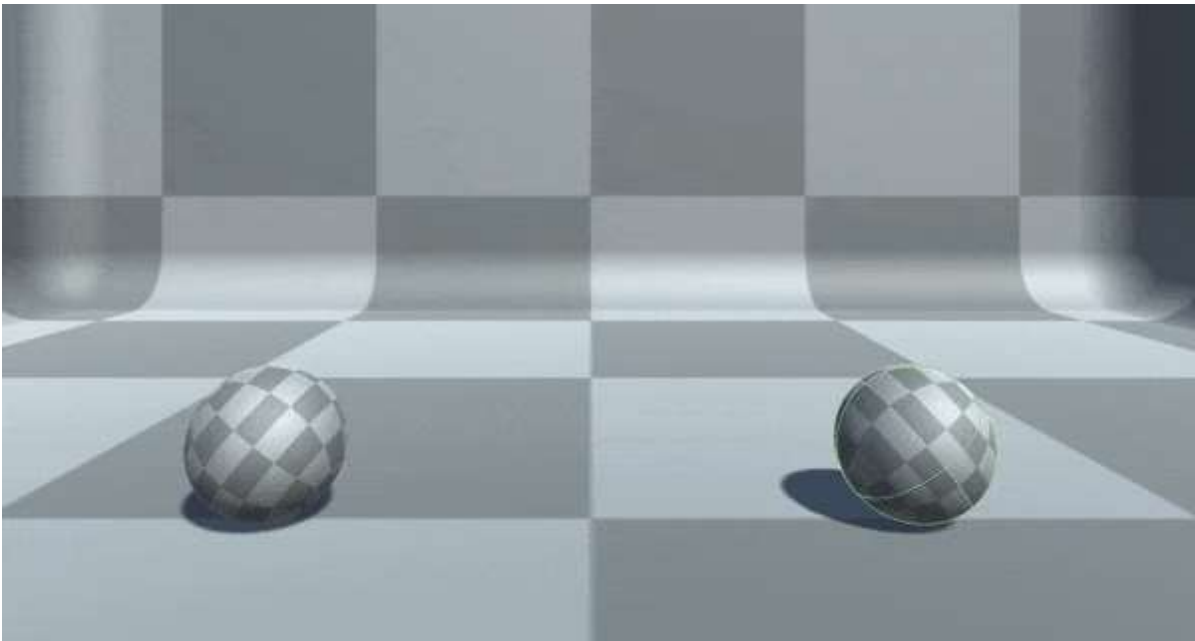
It's their relative mass that does, often referred to as their **mass ratio**. In other words: a pair of 20 kg colliding steel boxes would react the same way as a pair of 10 kg wooden boxes. However, it would be far harder for a 10 kg wooden box to move a 20 kg steel box than it would be for a 30 kg cast iron box.



Both rigidbody spheres have a mass of 10 kg.



Both rigidbody spheres have a mass of 20 kg (looks the same as the 10 kg ones).



The one on the right weights 20 kg, the one on the left 10 kg.

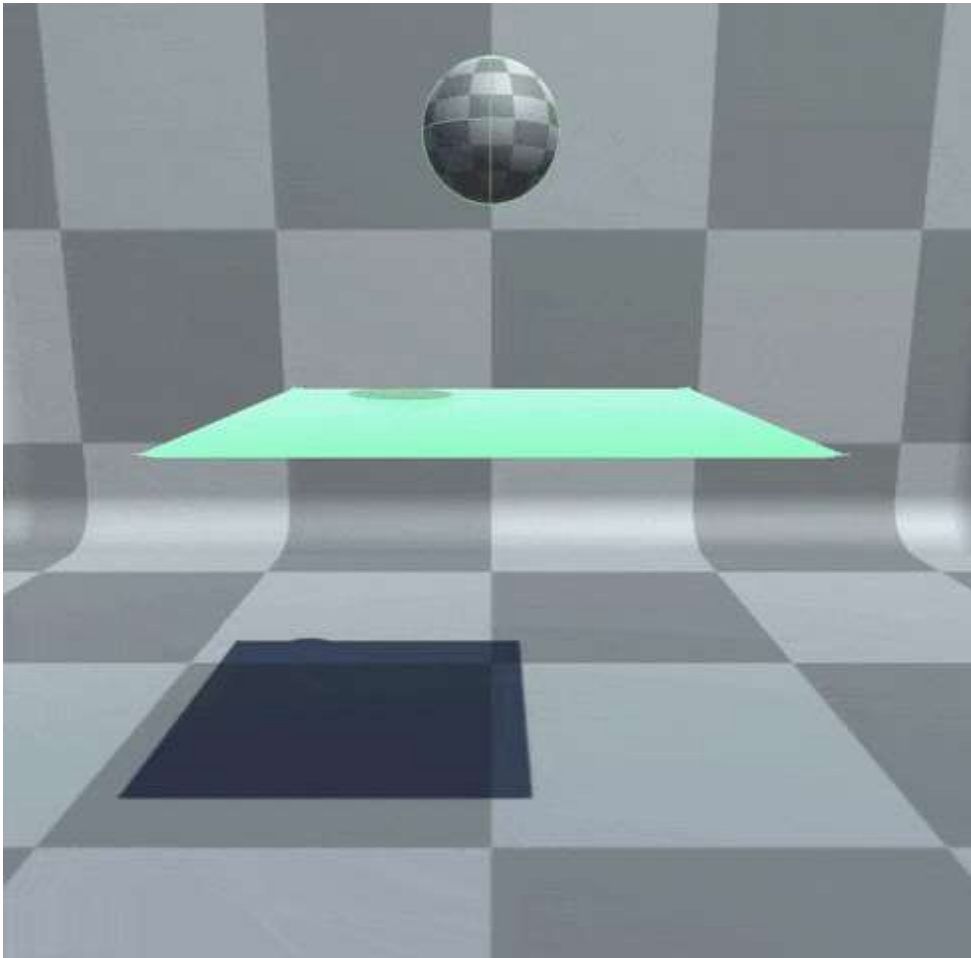
#3. Beware of large mass ratios

Iterative physics engines have a hard time with high mass ratios. As we discussed in the above section, the way Obi deals with many simultaneous constraints is by enforcing them all multiple times (iterating) and accumulating their effect. However when two objects involved in a constraint have a large mass ratio, the heavier one is displaced *a very small amount each time the constraint is enforced*, which means it may require a **lot of iterations** for it to meet all constraints simultaneously. So either you spend a lot of iterations and take a performance hit, or you live with partially solved and excessively stretchy constraints: for instance, ropes and cloth that stretch a lot when attached to / colliding with much heavier objects.

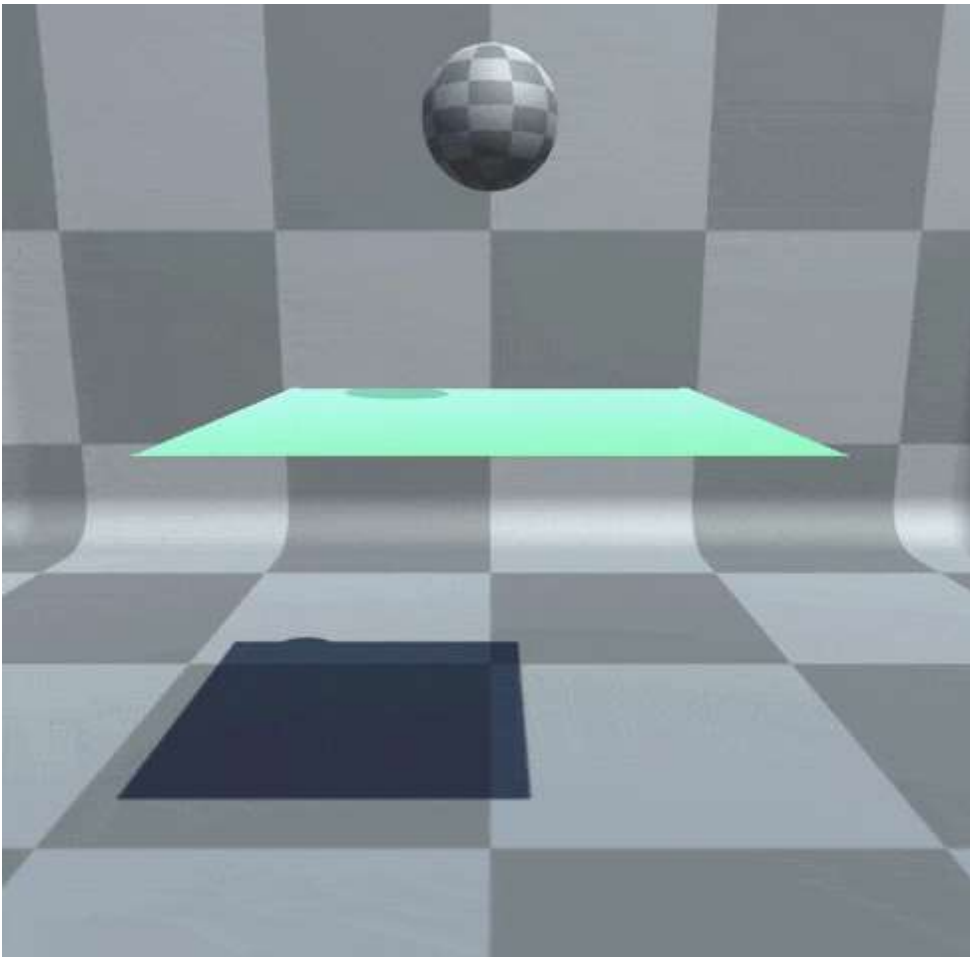
Obi uses PBD (position-based dynamics) which means constraints acts on **positions** while traditional constraints used in other engines act on **velocities**, but all existing realtime engines iterate over constraints while accumulating their effect on either position or velocity. As a result, the majority of game physics engines are affected by this: PhysX, Unity physics, Havok, Bullet, Box2D, Obi, etc. A rough guideline is that no two objects in your scene should exceed a mass ratio of **1:10**, that is, no potentially interacting object should be more than 10 times heavier than another. You can have higher mass ratios, but it will typically require you to increase your solver's budget (iterations/substeps) to ensure constraints are met.

Note there's other types of engine - reduced coordinate solvers, direct solvers, primal descent methods - that are unaffected by large mass ratios, but they have their own kind of limitations and are typically costlier.

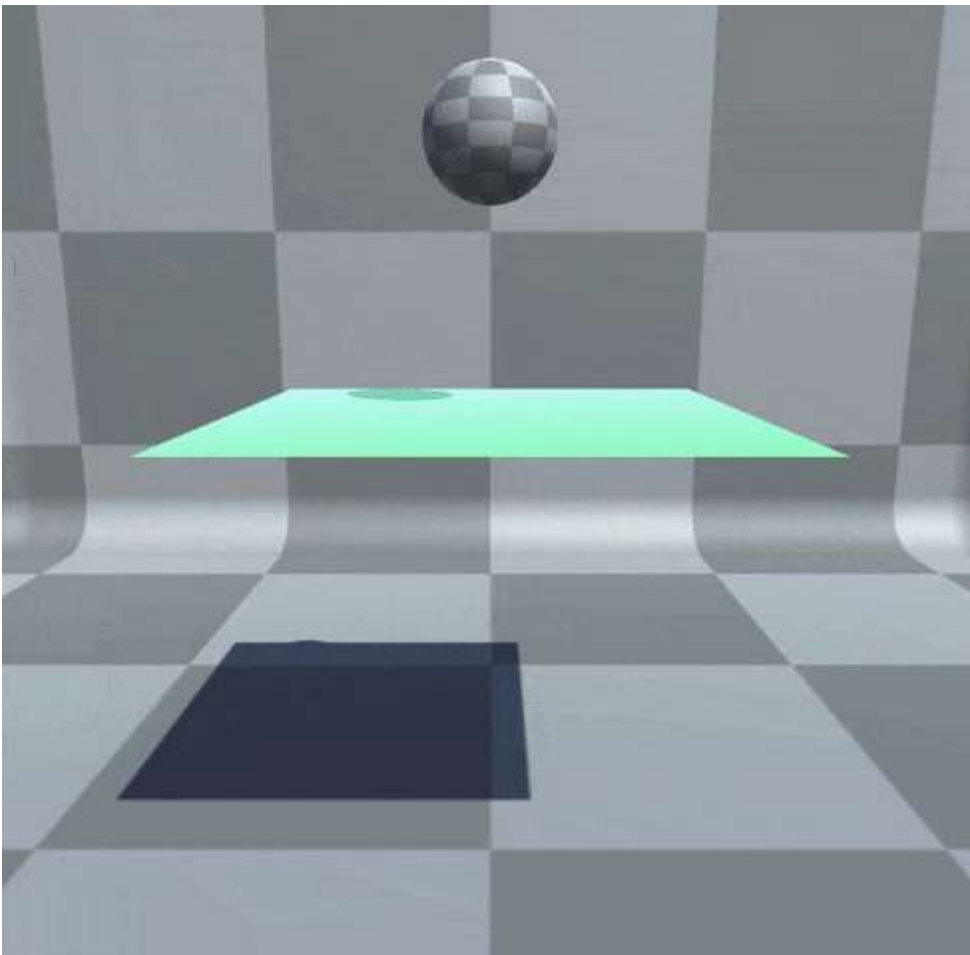
If you absolutely must deal with large mass ratios when using Obi, **instead of spending more iterations spend more substeps (obisolver.html)**. Substeps are significantly more effective at dealing with large mass ratios.



Cloth particles weight 100 grams each, they can easily stop and support a 10 kg rigidbody when using 4 substeps.



A 1000 kg rigidbody will easily push the particles out of its way if only using 4 substeps.



30 substeps are needed for 100 grams particles to support the same 1000 kg rigidbody.

Constraint types

Collision constraints



Collision constraints try to **keep particles outside of colliders**. High iteration counts will yield more robust collision detection when there are multiple collisions per particle.

Friction constraints



Collision constraints try to **reduce the tangential velocity of particles upon collision**. High iteration counts will yield more accurate friction calculations.

Particle collision constraints



Identical to collision constraints, but for when collisions happen between particles.

Particle friction constraints



Identical to friction constraints, but for when collisions happen between particles.

Distance constraints



Each distance constraint tries to keep two particles at a fixed distance from each other. These are responsible for the **elasticity of cloth and ropes**. High iteration counts will allow them to reach higher stiffnesses, so your ropes/cloth will be less stretchy.

Pin constraints



A pin constraint will apply forces to **a particle** and a **rigidbody** so that they maintain their relative position. They are created and used by dynamic attachments ([attachments.html](#)). High iteration counts will reduce the amount of drift at the pin location, making the attachment more robust.

Stitch constraints

Stitch constraints try to keep 2 particles on top of each other. They are created and used by stitchers (stitcher.html). High iteration counts will reduce the amount of drift at the stitch location, making it more robust.

Volume constraints

Each volume constraint takes a group of particles positioned at the vertices of a mesh, and tries to maintain the mesh volume. Used to **inflate cloth and create balloons**. High iteration counts will allow the balloons to reach higher pressures, and keep their shape more easily.

Aerodynamic constraints

This is the only type of constraint that doesn't have an iteration count. They are always applied only once, that is enough. Used to simulate **wind**.

Bend constraints

Each bend constraint will work on three particles, trying to get them in a straight line. These are used to make cloth and ropes **resistant to bending**. As with distance constraints, high iteration counts will allow them to reach higher stiffness.

Tether constraints

These constraints are used to **reduce stretching of cloth**, when increasing the amount of distance constraint iterations would be too expensive. Generally 1-4 iterations are enough for tether constraints.

Skin constraints

Skin constraints are used to keep **skeletally animated cloth close to its skinned shape**. They are mostly used for character clothing. Generally 1-2 iterations are enough for skin constraints.

Density constraints



Each density constraint tries to keep the amount of mass in the neighborhood around a particle constant. This will push out particles getting too close (as that would increase the mass) and pull in particles going away (which results in surface tension effects). Used to simulate **fluids**. High iteration counts will make the fluid more incompressible, so it will behave less like jelly.

Shape matching constraints



每个形状匹配约束记录一组粒子的静止形状，然后调整它们的位置，使它们尽可能地保持该形状。这些由**软体**使用。

拉伸/剪切约束



拉伸/剪切约束调整一对粒子沿参考系轴的位置，该轴由旋转四元数确定。**杆状体和骨骼**会使用到这些约束。

弯曲/扭曲约束



弯曲/扭曲约束用于调整一对粒子的方向，以防止弯曲和扭曲。**杆和骨骼**会使用这种约束。

链约束



链约束采用粒子列表，并尝试使用直接非迭代求解器来维持它们的总长度。这些约束由**杆**使用。